

KOBLY · INTERNAL HANDOFF DOCUMENT

The Kobly website, fully documented

Every page, component, data source and email template on kobly.no — written as a rebuild spec for Django + server-rendered HTML/SCSS/vanilla JS, no React/Next.js.

■ **Build order: the /wizard lead-capture form (\$5) is priority one. Everything else on this site exists to funnel a visitor into that form.**

Source analyzed exhaustively: all 10 routes under `app/` all ~34 components, all 7 `lib/` data modules, and all 4 transactional email templates.

Prepared: 2026-08-20

0 Contents

- 1 Executive summary & build priority

2 Full route map (every URL on the site)

3 Global design system (color, type, spacing, motion)

4 Global chrome (header, footer, mobile menu, shared building blocks)

5 ■ PRIORITY 1 — the lead form (`/wizard`)

6 Home page (`/`)

7 City pages (`/[city]`)

8 Oslo district pages (`/oslo/[bydel]`)

9 Agencies directory & profile (`/byraer` , `/byraer/[slug]`)

10 Blog (/blogg , /blogg/[slug])

11 Partner page (/partner)

12 Transactional emails (4 templates + internal preview tool)

13 Full data catalogs (cities, districts, agencies, articles)

14 SEO / sitemap behavior

15 Known quirks & open decisions

16 Django rebuild architecture & build order

1 Executive summary & build priority

Kobly is a Norwegian lead-generation marketplace for moving companies ("flyttebyråer"). The business model, stated repeatedly across the copy: a visitor fills out **one form** describing their move, Kobly routes that request to **three vetted agencies**, and the visitor compares offers and picks one — for free, with no obligation. Every marketing page on the site (home, city pages, district pages, agency profiles, blog posts, the partner page) exists to funnel traffic into that one form.

Build priority, as instructed: the `/wizard` form (§5) is step one, full stop — it's the only page that actually captures a lead. Sections 6–11 (home, city/district SEO pages, agency directory, blog, partner page) are marketing surface around it and can follow. Section 12 (transactional emails) is effectively part of the form's job too — submitting the form is supposed to trigger the "receipt" email, and matching/booking is supposed to trigger the other three — so budget for it alongside the form's backend, not as a separate later phase.

The second most important fact in this whole document, expanded in §5.2 and §15: **none of this is wired to a real backend today**. The Next.js app's wizard submit handler is a `console.log`. The four email templates in `lib/emails/` are beautifully finished, table-based, email-client-safe HTML — but nothing in the codebase actually sends them; they only exist to be looked at on an internal `/email-preview` page. Porting this site to Django means the front-end fidelity is a well-defined copy job, but the entire backend (lead storage, agency matching, email dispatch, the partner application flow) is new product/engineering work with only the front-end contract to go on.

2 Full route map

Route	Type	Purpose
/	Static	Home / marketing landing page
/wizard	Client-rendered, no server data	PRIORITY 1 the 5-step lead form
/[city]	Static params: oslo, bergen, trondheim, stavanger, tromso	City-specific SEO landing page
/oslo/[bydel]	Static params: 14 Oslo districts	District-specific SEO landing page (Oslo only)
/byraer	Static	Directory of all partner agencies
/byraer/[slug]	Static params: 4 agencies	Individual agency profile page
/blogg	Static	Blog index
/blogg/[slug]	Static params: 3 articles	Individual article page
/partner	Static	B2B landing page for moving companies to sign up
/email-preview	Internal tool, robots: noindex	Renders all 4 transactional emails inline for review
/email-preview/[id]/raw	API route (GET, returns raw HTML)	Serves one email template's finished HTML, e.g. for pasting into an ESP
/sitemap.xml	Generated (app/sitemap.ts)	Lists all static + dynamic URLs for search engines

Every route uses the same root layout (app/layout.tsx): loads three fonts (see §3.2) and sets the page `<html lang="nb">` — the entire site is Norwegian (Bokmål), no i18n/locale switching exists anywhere.

3 Global design system

One shared token set (`app/globals.css`) drives every page — wizard included. Reproduce this once in Django's base SCSS and every page/template pulls from it.

3.1 Color tokens

--bg

#f4f1ea

--surface

#ffffff

--surface-soft

#faf9f6

--ink

#1a1a1a

--ink-muted

#5c5c5c

--line

#e6e1d6

--brand (primary!)

#221814

--brand-ink

#faf9f6

--accent-lime

#d6e8a8

--accent-amber

#e8a87c

--move (defined, rarely used)

#2f5fa8

The **"brand" color is near-black brown** (`#221814`), used as every primary CTA fill, every "selected" state, and the footer background — not a blue/green corporate color. Text hierarchy is built almost entirely from opacity on `--ink` (`ink-muted #5c5c5c` $\approx$ 70% opacity black, plus ad-hoc `text-ink/40` , `/45` , `/50` , `/60` Tailwind opacity utilities) rather than a large palette of distinct grays. One hardcoded color recurs outside the token list: `#3D5507` (dark olive) — used for checkmark icons and the "to" map pin.

3.2 Typography

Token	Family	Source	Used for
--font-sans (default)	Moderat	Local <code>.otf</code> , weights 400 & 600 only	All body text, nav, buttons, most headings on marketing pages
--font-serif / --font-heading	Crimson Pro	Google Font	Wizard step headings only (<code>font-serif</code> explicitly applied); marketing H1/H2s mostly use plain sans at heavier weight instead
--font-display	Syne	Google Font	Loaded sitewide, not actually applied anywhere in the current markup

`font-feature-settings: "cv11"` is on globally; `ss01` is deliberately left off because it swaps `?` & `%` glyphs for alternates missing from the Semibold weight, making punctuation fall back to Regular and look thin next to bold text. Keep this in the Django SCSS.

3.3 Shape, spacing, elevation, motion

Token	Value	Where
Card radius	14px	Nearly every card, tile, image, and input sitewide — the single most repeated shape token
Pill radius	9999px (full)	All buttons/CTAs, nav pills, filter chips
Section rhythm	<code>py-20</code> mobile → <code>py-28</code> tablet → <code>py-32</code> / <code>py-40</code> desktop	Every marketing section wrapper

Content width	<code>max-w-6xl</code> (72rem) standard; <code>max-w-4xl</code> for FAQ; <code>max-w-[680px]</code> for article/about body text	All marketing sections
Card shadow (elevated)	0 20px 60px rgba(0,0,0,.28), 0 4px 16px rgba(0,0,0,.10)	Wizard card only
Card shadow (soft/hero)	0 1px 2px rgba(0,0,0,.03), 0 12px 32px -16px rgba(0,0,0,.10)	<code>HeroCard</code>
Hover lift	icon translates +2px/-2px on hover (ArrowUpRight pattern)	List/card links sitewide (city list, district list, agency cards)
Marquee	100s linear infinite scroll, seamless loop via 4x tile duplication + <code>translateX(-50%)</code>	<code>ImageStrip</code> on home page
Wizard transitions	260ms slide+fade (step content), 400ms crossfade (mobile background)	<code>/wizard</code> only, see §5

3.4 Icon set

All icons sitewide are `lucide-react` — simple 24×24 stroke SVGs, `currentColor`. Full inventory used across the site: `ArrowRight`, `ArrowUpRight`, `ArrowLeft`, `Check`, `ChevronRight`, `Info`, `MapPin`, `Mail`, `Menu`, `Plus`, `Minus`, `Search`, `Send`, `Star`, `Upload`, `X`, `ZoomIn`, `ZoomOut`, `Crosshair`, `LocateFixed`, `Filter`, `Wallet`, `CircleCheck`, `Sparkles`. Since Lucide ships raw SVG paths, port these as static inline `<svg>` markup or an SVG sprite in Django templates — no JS icon library needed.

4 Global chrome

Present on every marketing page except `/wizard` (which is a fullscreen takeover with its own minimal chrome) and `/email-preview` (standalone internal tool).

4.1 Header

Sticky top nav, transparent until scrolled >8px then `bg-bg/85 backdrop-blur-md`. Left: Logo (links `/`). Desktop (≥ 1024 px, Tailwind `lg:`): inline nav — **Blogg, Byråer, For flyttebyråer** — plus a brand-filled pill CTA **"Få 3 tilbud gratis"** linking `/wizard`. Mobile: nav collapses to a single "Meny" text button that opens a fullscreen overlay menu.

4.2 Mobile menu (fullscreen overlay)

Reuses the same `FullscreenOverlay` primitive as the wizard's mobile map picker (locks body scroll, closes on Escape, `role="dialog"`, 260ms fade-in). Header row: logo + "Lukk" (close) button. Body: the 3 nav links as large serif headings (`text-4xl font-serif`), stacked, centered vertically. Bottom: full-width brand CTA "Få 3 tilbud gratis" → `/wizard`.

4.3 Footer

Full-bleed `bg-brand` (near-black) section. Left: Logo (light tone) + one-line company blurb + a low-key "E-post" link to the internal `/email-preview` tool (⚠ this is an *internal QA link left in the production footer* — decide whether to keep, hide behind auth, or drop it in the rebuild) + "Kobly AS · © 2026". Right: a light-tinted card, "Klar for å flytte?" + full-width CTA to `/wizard`. Bottom: a giant (up to 24rem), near-invisible (8% opacity) watermark word "kobly" spanning the full footer width, purely decorative.

4.4 Shared building blocks used across many pages

Component	What it is	Used on
<code>HeroCard</code>	The little white "3 icons → CTA" card (Kobly mark · dots · destination icon, headline, body, full-width pill CTA). Accepts an <code>endVisual</code> override (used to show an agency's logo instead of the default box icon on agency pages).	Home hero, city hero, district hero, agency profile hero (desktop only in all cases)
<code>Stats</code>	4-tile stat grid: 100% gratis · 1 min å fylle ut · 5 000+ fornøyde kunder · 4.8/5 vurdering. Has a <code>compact</code> variant (smaller, translucent tiles) used inline over the home page's image marquee.	Home (compact, over marquee), city pages, district pages, partner page (different numbers, see §11)
<code>HowItWorks</code>	3-step numbered card row: Fyll ut skjema → Vi kobler deg med de beste → Velg tilbud, each with a custom illustration PNG, ending in a CTA to <code>/wizard</code> .	Home, city pages, district pages
<code>Services</code>	Static 6-card grid: Flyttehjelp, Flyttevask, Lagring, Utlandsflytting, Kontorflytting, Dødsbo — same six everywhere, not filtered per city/agency.	Home, city pages, district pages
<code>FAQ</code>	Accordion, single-open-at-a-time, <code>Plus / Minus</code> icon toggle, 180° rotation on open.	Home, city pages, district pages (same 5 Q&As everywhere); Partner page has its own separate 5-Q&A set (<code>PartnerFAQ</code>)
<code>FooterCTA</code>	Centered "Klar for å flytte?" heading + <code>PostnummerInput</code> .	Home, blog index, agency directory (as the section right before the real <code>Footer</code>)
<code>PostnummerInput</code>	A 4-digit postal-code text field + submit pill. Validates <code>/^\d{4}\$/</code> client-side, shows inline error text if touched+invalid,	FooterCTA only

	on submit does <code>router.push('/wizard?fra=' + value)</code> .	
WizardCTA	Inline brand-filled callout block ("Skal du flytte?" + CTA), used <i>mid-content</i> not just at page end.	Blog article body (as a <code>{type:"cta"}</code> block) + always appended once at the very end of every article

Functional gap worth deciding on for the rebuild: `PostnummerInput` collects a 4-digit *postal code* and passes it as `?fra=1170` . The wizard reads `fra` straight into its free-text "Fra adresse" field with no lookup — so a user who used the postal-code box lands on step 1 of the wizard with `"1170"` sitting in the address field as literal text, not resolved to a real address or even validated as a real Norwegian postal code. Either keep this exact (low-fidelity) behavior or use it as an opportunity to actually resolve the postcode server-side in Django.

5 PRIORITY 1 — The lead form (/wizard)

This is the one page that captures a lead — build it first, and build it exactly. Full behavioral, visual, and data spec follows; this section is self-contained enough to hand to a developer on its own.

5.1 What it is

A fullscreen, 5-step wizard collecting a moving request: origin/destination, move type, home size, date, inventory, and contact details — then (per copy) routes it to three moving agencies who reply by email within ~24 hours. Supports pre-fill via query string: `?fra=<text>` and `?by=oslo|bergen|trondheim|stavanger|tromso` (city keys from `lib/cities.ts`) to center the map before any typing happens. State lives in one in-memory JS object for the whole session — **no persistence**; a page refresh resets to step 1.

5.2 Critical: there is no backend today

The entire submit handler is:

```
const submit = () => {
  console.log("Wizard submitted:", data);
  setSubmitted(true);
};
```

No `/api/*` route exists. No email is sent. No database write happens. Uploaded photos never leave the browser (they're `URL.createObjectURL blob` previews only). **Everything past "validate and disable/enable the Neste button" is new work** for Django — this document describes the contract (what fields must exist, what "valid" means) not an existing implementation to port.

5.3 Page shell & responsive layout

- **Shell:** `fixed inset-0 full-viewport overlay`, independent of normal page scroll. Centered white card, `max-width:1060px`, `width:calc(100% - 3rem)`, `border-radius:14px`, background `--surface-soft`, heavy shadow (§3.3), `min-height:640px` on desktop.
- **Desktop ($\geq 1024\text{px}$):** card splits into 2 flex columns — left (58%) = the form; right (flex-1) = a live Leaflet map (step 1), a static step photo (steps 2–4), or a live summary/receipt panel (step 5). Page background: fixed `boxes-and-plants.jpg` photo washed under 85%-opacity `--bg`. Logo floats top-center, absolutely positioned, links `/`.
- **Mobile ($< 1024\text{px}$):** right column hidden entirely. Full-page background becomes a step-specific photo + dark gradient scrim (`from-black/75 via-black/55 to-black/70`), crossfading 400ms on step change (old image stays mounted under the new one, which fades in over 260ms, then the old one unmounts). Logo sits inline above the card instead of overlaid on a photo.

Mobile background image map, with 2 deliberate exceptions (steps 1 & 5 share the calm "boxes-and-plants" image since they have no matching desktop lifestyle photo; step 4 substitutes step 1's photo on mobile because its real desktop photo, `R1-09476-0028.jpg`, is too dark — avg. brightness 39/255 — behind the scrim):

```
step 1 → boxes-and-plants.jpg    step 2 → R1-07829-0034.jpg
step 3 → rull3_26.jpg            step 4 → R1-09476-0023-kopi.jpg (NOT -0028.jpg)
step 5 → boxes-and-plants.jpg
```

Desktop right-column art (steps 2–4 only): 1→`R1-09476-0023-kopi.jpg`, 2→`R1-07829-0034.jpg`, 3→`rull3_26.jpg`, 4→`R1-09476-0028.jpg`.

Left column, top to bottom: 5-segment progress bar (filled = completed steps only, current step not counted as filled), "Step X av 5" label, animated step content, and a bottom nav bar (absolute-positioned) with "Tilbake" (back, text link, hidden on step 1) left and the primary button right.

5.4 Full state shape

```
type WizardData = {
  flytteType: "privat" | "bedrift" | "internasjonal" | "";
  fra: string; fraCoord: { lat: number; lon: number } | null;
  til: string; tilCoord: { lat: number; lon: number } | null;
  boligtype: "leilighet" | "rekkehus" | "enebolig" | "annet" | "";
  flyttedato: string; // yyyy-mm-dd or ""
  fleksibel: boolean;
  beskrivelse: string;
  bilder: File[]; // client-only today – never uploaded anywhere
  navn: string; telefon: string; epost: string;
};
```

This is the exact payload shape the Django form/model needs to accept (plus photo files as multipart uploads). Every field starts empty; each step writes directly into this one object incrementally — there is no per-step sub-save.

5.5 Step 1 — "Hvor skal du flytte?" (address + map)

Subtitle: *Skriv inn fra-adresse og til-adresse.* Valid to advance: both `fra` and `til` >2 characters (coordinates NOT required).

Address autocomplete (`AddressField`): 200ms-debounced calls to `GET https://ws.geonorge.no/adresser/v1/sok?sok={q}&treffPerSide=8&side=0` (Kartverket's free, keyless official Norwegian address registry), up to 8 results in a dropdown (address bold, postcode+city muted underneath). Selecting one fills text as `"{adresstekst}, {postnummer} {poststed}"` and stores its lat/lon. The "Fra" field only also shows a pinned "Bruk min plassering" (use my location) row using `navigator.geolocation`, reverse-geocoded (see below) with fallback label "Min posisjon". Manual typing clears any attached coordinate. Dropdown closes 120ms after blur (to let a click register first).

Map — Leaflet + free CARTO Voyager raster tiles (`{s}.basemaps.cartocdn.com/rastertiles/voyager/{z}/{x}/{y}{r}.png`), custom zoom/locate controls (native controls disabled). Reverse-geocode endpoint: `GET https://ws.geonorge.no/adresser/v1/punktsok?radius=200&lat={lat}&lon={lon}&treffPerSide=1&side=0`.

- **Desktop** (`MapPanel` , right column): zoom in/out buttons, "use my location" button (pulses while locating), "Plasser fra"/"Plasser til" pill toggles that arm crosshair "placing mode" — next map click drops/reverse-geocodes that pin. Draggable pin markers (from = dark `#221814` , to = olive `#3D5507`), a 250m translucent circle around "from", a dashed connecting line with a small Kobly ring-logo icon at the midpoint once both pins exist, auto `fitBounds` (40px padding, max zoom 13). Floating chip shows "Ca. {fromStreet} → {toStreet}". Bottom-right attribution tooltip "© Leaflet · © CARTO · © OpenStreetMap" (required by CARTO's free tier terms, even with default controls hidden).
- **Mobile** (`MapPickerOverlay` , opened via a "Plasser i kart" pill under each address field): fullscreen modal, fixed center crosshair pin, user pans the *map* underneath it. Reverse-geocodes the map's current center on open and on every `moveend` , shown in a bottom sheet ("Valgt plassering" / address, or "Finner adresse ..." dimmed while loading). Confirm button: full-width brand pill "Bruk denne plasseringen". The trigger pill itself changes to a lime-tinted "Plasseringen valgt · endre i kart" with a checkmark once a pin exists.

5.6 Step 2 — "Hva slags flytting er det?" (type & size)

No subtitle. Two required single-select pill groups (`PillButton` : unselected = light tan `#F3EEE3` , selected = solid brand fill + cream text):

Type (3-col grid)

Privat · Bedrift · Internasjonal

Størrelse (2-col grid)

Leilighet · Rekkehus · Enebolig · Annet

5.7 Step 3 — "Når skal du flytte?" (date)

Subtitle: *Velg en dato eller la oss vite om du er fleksibel.* A native `<input type="date">` ("Ønsket flyttedato") and a dashed-pill toggle "Jeg er fleksibel på datoen" are mutually exclusive — picking a date clears flexible, toggling flexible on clears/disables (50% opacity) the date field. Valid: date set **or** flexible=true.

5.8 Step 4 — "Hva skal du flytte?" (goods/photos)

Subtitle: *Legg til bilder eller en beskrivelse av tingene dine.* Always valid (optional step). A 4-row textarea ("Beskrivelse", placeholder "F.eks. 3-seters sofa, stort spisebord, 2 senger, 10 esker...") and a 4-column square-thumbnail photo grid (each with an "x" remove button, last cell is always a dashed "Last opp" upload tile, `<input type="file" multiple accept="image/*">`). New files append to the existing array. Previews are blob URLs, explicitly revoked on unmount — files stay in memory only, no upload step exists today.

5.9 Step 5 — "La oss ta kontakt" (contact) + live summary

Subtitle: *Vi kobler deg med tre byråer. Du hører fra dem innen 24 timer.*

Field	Placeholder	Validation
Navn	Ola Nordmann	<code>trim().length > 1</code>
Telefon	+47 000 00 000	<code>/^[\\d\\s+]{8,}\$/</code>
E-post	ola@eksempel.no	<code>/\\S+@\\S+\\.\\S+/</code> (permissive, not RFC-strict)

Button reads **"Send forespørsel"** here instead of "Neste". Desktop right column on this step only is a *live* receipt (`Summary` , re-renders from the same data in real time): "Kvittering" / "Din forespørsel" / "Vi sender denne til tre kvalitets sjekkede byråer i ditt område." + conditional rows (Fra, Til, Flytting [type · size], Når [formatted date or "Fleksibel dato"], Innhold, photo thumbnails).

5.10 Thank-you screen

Replaces the wizard entirely once submitted (not step 6 — a separate render branch). Same darkened background photo (flat 65% black, no gradient). Contents: a staggered pop-in "routing" illustration (Kobly ring-logo circle → 3 dots → 3 lime checkmark squares, 120/230/340ms bouncy delays), heading **"Forespørselen er sendt!"**, body copy about the 3 agencies, full-width CTA "Tilbake til forsiden" → `/`.

5.11 Navigation & motion rules

- "Neste"/"Tilbake" are the only navigation — no clickable progress segments, no swipe.
- Primary button renders `disabled` (gray, no hover) whenever the step is invalid — no error messages/shake, just the disabled affordance.
- Forward navigation slides new content in from the right, back from the left (260ms ease + opacity, both directions use the same easing). The right-column art/map/summary panel does an independent 260ms fade only (no slide), keyed on step number.
- Nothing persists — no localStorage, no URL step param, no session. A refresh loses all progress. Decide explicitly whether the Django rebuild should add session-backed persistence, since a lost-progress refresh feels worse outside a SPA.

5.12 Exact copy (verbatim, for pixel/word parity)

```
Progress label:      "Steg {n} av 5"

STEP 1  H1 "Hvor skal du flytte?"  ·  Sub "Skriv inn fra-adresse og til-adresse"
Fields: "Fra adresse" (F.eks. Kongens gate 1, 0153 Oslo) / "Til adresse" (F.eks. Storgata 14, 0184 Oslo)
"Bruk min plassering"  ·  Map picker: "Plasser i kart" / "Plasseringen valgt · endre i kart"
Map overlay titles: "Hvor flytter du fra?" / "Hvor flytter du til?"
Map overlay hint: "Flytt kartet så nålen står på adressen din"  ·  Confirm: "Bruk denne plasseringen"
```

```
Map overlay label: "Valgt plassering" / loading "Finner adresse ..."  
Desktop map pills: "Plasser fra" / "Plasser til" / armed: "Klikk på kartet" · Chip prefix: "Ca."  
Attribution: "© Leaflet · © CARTO · © OpenStreetMap"  
  
STEP 2 H1 "Hva slags flytting er det?"  
"Type": Privat / Bedrift / Internasjonal · "Størrelse": Leilighet / Rekkehus / Enebolig / Annet  
  
STEP 3 H1 "Når skal du flytte?" · Sub "Velg en dato eller la oss vite om du er fleksibel"  
"Ønsket flyttedato" · "Jeg er fleksibel på datoen"  
  
STEP 4 H1 "Hva skal du flytte?" · Sub "Legg til bilder eller en beskrivelse av tingene dine."  
"Beskrivelse" (F.eks. 3-seters sofa, stort spisebord, 2 senger, 10 esker...)  
"Bilder" · "Legg ved bilder av tingene dine for et mer presist tilbud." · "Last opp"  
  
STEP 5 H1 "La oss ta kontakt" · Sub "Vi kobler deg med tre byråer. Du hører fra dem innen 24 timer."  
"Navn" (Ola Nordmann) · "Telefon" (+47 000 00 000) · "E-post" (ola@eksempel.no)  
Submit: "Send forespørsel"  
  
SUMMARY (desktop, step 5) "Kvittering" / "Din forespørsel" /  
"Vi sender denne til tre kvalitetssjekkede byråer i ditt område."  
Rows: Fra / Til / Flytting / Når / Innhold / Bilder · flexible value: "Fleksibel dato"  
  
NAV "Tilbake" · "Neste"  
  
THANK-YOU "Forespørselen er sendt!"  
"Vi har nå sendt forespørselen din til tre kvalitetssjekkede byråer i ditt område. Du mottar tilbud på  
e-post innen kort tid."  
"Tilbake til forsiden"
```

5.13 External APIs used (client-side, free, keyless)

Purpose	Endpoint
Address autocomplete	GET ws.geonorge.no/adresser/v1/sok?sok={q}&treffPerSide=8&side=0
Reverse geocode	GET ws.geonorge.no/adresser/v1/punktsok?radius=200&lat={lat}&lon={lon}&treffPerSide=1&side=0
Map tiles	{s}.basemaps.cartocdn.com/rastertiles/voyager/{z}/{x}/{y}{r}.png (CARTO Voyager, attribution required)

All CORS-open — the vanilla-JS port calls them identically client-side; Django doesn't need to proxy them unless you want server-side caching/rate-limiting.

5.14 Fonts & images specific to this page

```
Fonts: Moderat-Regular.otf, Moderat-Semibold.otf (must be re-licensed/self-hosted), Crimson Pro (Google, free)  
Images: boxes-and-plants.jpg, R1-09476-0023-kopi.jpg, R1-07829-0034.jpg, rull3_26.jpg, R1-09476-0028.jpg  
Icons (lucide): ArrowRight, Check, Info, MapPin, Send, Upload, X, ZoomIn, ZoomOut, Crosshair, LocateFixed
```

6 Home page (/)

Section order top to bottom: Header → Hero → ImageStrip (with compact Stats overlaid) → HowItWorks → Testimonials → Trust → Services → Cities → FAQ → FooterCTA → Footer .

Section	Content
Hero	H1 "Vi finner det beste flyttebyrået for deg". Mobile: eyebrow hidden, short subtitle + CTA pill shown below H1. Desktop: eyebrow "Norges smarteste tilbudstjeneste" shown above H1, no subtitle/CTA (a HeroCard sits in the right column instead, 1.1fr/0.9fr grid split).
ImageStrip	Infinite horizontal marquee (100s linear loop) of 5 lifestyle photos, tiled ×4 for seamless wrap, aspect 4:5, heights 340→440→540px responsive, edge-masked (fades to transparent at both ends on ≥640px). A compact 4-tile Stats row sits below it: 100% gratis · 1 min å fylle ut · 5 000+ fornøyde kunder · 4.8/5 vurdering.
HowItWorks	H2 "Slik fungerer det". 3 numbered cards: (1) Fyll ut skjema / Svar på noen enkle spørsmål. Tar under to minutter. (2) Vi kobler deg med de beste / Tre kvalitetssjekkede byråer kobles med din flytting. (3) Velg tilbud / Sammenlign, velg ditt byrå, og avtal resten direkte med dem. Each card has a custom PNG illustration. Ends in a CTA to /wizard .
Testimonials	H2 "Hva sier kundene våre?" 3 quote cards with photo, quote, name · role: Kristoffer Skogen (Privatkunde, Bergen), Sunniva Thune (Næringsflytting, Oslo), Eivind Johansen (Privatkunde, Trondheim).
Trust	H2 "Norges smarteste tilbudstjeneste" (same headline text reused from the desktop hero eyebrow). Body about Kobly not moving things itself. Two CTAs: "Få 3 tilbud gratis" (brand) + "Møt byråene" (outline, → /byraer). Below: all 4 agency logos in a row, each linking to its /byraer/[slug] profile.
Services	H2 "Alt du trenger på ett sted" + amber underline accent. 6 static cards: Flyttehjelp, Flyttevask, Lagring, Utlandsflytting, Kontorflytting, Dødsbo (full body copy in code, §4.4 table references this same component reused on city/district pages).
Cities	H2 "Flytter du i en av disse byene?" — 5 city cards (Oslo, Bergen, Trondheim, Stavanger, Tromsø) each linking /{slug} .
FAQ	Same shared 5-question accordion as city/district pages (see §4.4).
FooterCTA	H2 "Klar for å flytte?" + PostnummerInput .

7 City pages (`/[city]`)

Statically generated for 5 cities: `oslo, bergen, trondheim, stavanger, tromso` (source: §13.1). Unknown slug → `notFound()` (404). Metadata: `"Flyttebyrå i {name} – Kobly"`.

Section order: `Header` → `CityHero` → `Stats` (full, not compact) → `HowItWorks` → `Services` → `[DistrictList, Oslo only]` → `FAQ` → `FooterCTA` → `Footer`.

CityHero: eyebrow `"Flytting i {city}"`, H1 `"Vi finner det beste flyttebyrået for deg i {city}"`, body `"Få tre uforpliktende tilbud fra kvalitetssjekkede byråer som jobber i {city} og omegn."` CTA links carry the city: `/wizard?by={slug}` (this is what centers the wizard's map on that city per §5.5/§5.1). Mobile also shows a secondary `"Se andre byer"` link → `/`. Desktop shows a `HeroCard` in the right column with a city-specific body line.

Oslo-only: the Oslo page additionally renders `DistrictList` (links to all 14 district pages) between `Services` and `FAQ` — the only city with sub-pages.

8 Oslo district pages (`/oslo/[bydel]`)

Statically generated for all 14 districts (full list + copy in §13.2). Unknown slug → 404. Metadata: `"Flyttebyrå i {name} – få 3 tilbud gratis | Kobly"`, description = the district's own `metaDescription` field.

Structure: breadcrumb (`Kobly > Oslo > {District}`) → `DistrictHero` → a unique descriptive paragraph (`district.body` , distinct per district, exists specifically so these pages don't read as thin/duplicate content for SEO) → `Stats` → `HowItWorks` → `Services` → `DistrictList` (titled "Andre bydeler i Oslo", excludes the current one) → `FAQ` → `FooterCTA` .

DistrictHero mirrors CityHero's layout exactly but H1 is `"Flyttebyrå i {name} – få 3 tilbud gratis"` (SEO-formulated, different pattern from the city H1) and body is the district's own `lead` line. All CTAs on this page point to `/wizard?by=oslo` (the wizard has no per-district map center — it only knows city-level centers, so every district funnels into the same Oslo-centered map).

9 Agencies directory & profile

9.1 Directory (`/byraer`)

H1 "Bli kjent med byråene" + intro paragraph about the vetting process (Brønnøysund registration, liability insurance, tax certificate, ongoing review monitoring). A responsive 2-col grid of agency cards (`AGENCIES` , full data in §13.3), each card: logo, name, short description, star rating + review count, jobs-completed count, up to 4 coverage-area chips (+ "N til" overflow indicator), linking to `/byraer/{slug}` . Ends with the shared `FooterCTA` .

9.2 Agency profile (`/byraer/[slug]`)

Statically generated for the 4 agencies. Metadata: `"{name} – flyttebyrå på Kobly"` . Sections top to bottom:

1. **Hero** — back-link "Alle byråer", H1 = agency name, tagline, a row of 4 badge chips (jobs completed, rating ★/reviews, response time, "Medlem siden {year}"). Desktop right column: a `HeroCard` with `title=null` (so it leads straight into body copy) and the agency's own logo as the `endVisual` replacing the default icon.
2. **Om oss** — the agency's "about" copy, written in first person (2–3 paragraphs, full text in §13.3).
3. **Dette hjelper vi med** — services chips, each with a check icon.
4. **Hva kundene sier** — review grid (2 cols on md+), each card: stars, date, comment, `"{name} · {service}"`. 4 reviews per agency (full text in §13.3).
5. **Hvor vi jobber** — coverage-area chips with a map-pin icon.
6. **CTA band** — brand-filled block: "Klar for å hente inn tilbud?" + "Fyll ut én forespørsel, så går den til {agency} og to andre byråer..." + CTA "Få 3 tilbud gratis" → `/wizard` (no agency identifier is passed through — the wizard has no concept of a preferred/pre-selected agency today).

Mobile-only: a sticky bottom bar (`fixed inset-x-0 bottom-0` , backdrop-blur) duplicating the CTA as "Få tilbud fra {agency} og 2 andre byråer" while scrolling, hidden on desktop.

10 Blog

10.1 Index (/blogg)

H1 "Blogg" + intro. Responsive grid (1/2/3 cols) of article cards (image, date · read-time, title, 3-line clamped excerpt), sourced from `ARTICLES` (§13.4). Ends with `FooterCTA`.

10.2 Article (/blogg/[slug])

Statically generated for 3 articles. Back-link "Alle artikler" → date · read-time → H1 → ingress → full-width header image (16:9) → a rich-text body built from a typed block union:

```
type ArticleBlock =  
  | { type: "h2"; text }  
  | { type: "p"; text }  
  | { type: "list"; items: string[] }  
  | { type: "image"; src; alt; caption? }  
  | { type: "cta" }; // renders the WizardCTA inline, mid-article
```

Each article has one `{type:"cta"}` block embedded partway through the body, plus a second, unconditional `WizardCTA` always appended at the very end. This block-based content model is worth copying directly into Django — e.g. as a Wagtail StreamField or a simple ordered JSON/array field on an Article model — rather than a single rich-text blob, so the "insert a lead-gen CTA mid-article" pattern survives the port.

11 Partner page (/partner)

B2B landing page — the flip side of the funnel, aimed at moving companies who want to *receive* leads, not consumers submitting them. Metadata: "Bli partner — Kobly". Section order: Header → PartnerHero → PartnerStats → PartnerValueProps → PartnerHowItWorks → PartnerFAQ → PartnerCTA → Footer .

Section	Content
PartnerHero	Eyebrow "For flyttebyråer", H1 "Få flere kunder uten å konkurrere på pris", body about receiving qualified leads and only paying for jobs won. Two anchor-link CTAs: "Bli partner" (→ #bli-partner) and "Slik fungerer det" (→ #hvordan) — same-page scroll, not separate routes.
PartnerStats	5 000+ forespørsler i året · 70% booking-rate på leads · 0 kr oppstartskostnad · 24t snitt responstid.
PartnerValueProps	4 cards with icons: Forhåndskvalifiserte leads (Filter icon), Pay-per-job (Wallet), Du velger jobbene (CircleCheck), Bygg omdømme (Sparkles). Full body copy in source.
PartnerHowItWorks (id="hvordan")	3 steps: Søk om partnerskap → Få leads automatisk → Vinn jobber, betal per kontrakt.
PartnerFAQ	Its own separate 5-question accordion (cost, requirements, declining a lead, lead volume, time-to-first-lead) — NOT the same content as the consumer-facing FAQ.
PartnerCTA (id="bli-partner")	"Bli en av Norges mest foretrukne byråer" + a mailto:partner@kobly.no?subject=Søknad%20om%20partnerskap link as the entire application mechanism — there is no partner application form/database anywhere in the codebase , just a mailto link. This is a second, separate lead-capture flow to design for Django if partner onboarding is in scope.

12 Transactional emails

Four finished, production-quality HTML email templates exist in `lib/emails/` — table-based, fully inlined CSS, 600px max width, system fonts only, MSO/Outlook conditional comments, no external CSS or JS. **They are directly portable as-is** — this is pure server-side string-building (functions that return an HTML string), zero React involved, so translating to Django (Python f-strings, Jinja/Django templates, or a Python email library) is close to a line-for-line port, not a rewrite. None of them are wired to an actual send trigger today — they only render on the internal `/email-preview` page and its raw-HTML API route (`/email-preview/[id]/raw?base=<url>`), which makes image `src` s absolute for real delivery).

Template id	Trigger (intended)	Subject
receipt	To customer, immediately after wizard submit	"Vi har mottatt flytteforespørselen din"
new-lead	To an agency, when a request is matched to them	"Ny flytteforespørsel i Oslo – svar innen 24 timer"
offer	To customer, when an agency has sent a quote	"Du har fått et tilbud fra {agency}"
booking-confirmed	To customer, when they've picked an agency	"Flyttingen din er bekreftet med {agency}"

All four share one visual language via `lib/emails/layout.ts` 's reusable building blocks — port these as Django template partials/macros:

- `emailShell()` — the outer wrapper: centered logo, white content card with border+radius on the `--bg` color, footer with company address ("Kobly AS · Torggata 15, 0181 Oslo"), contact (`hei@kobly.no` / 21 09 88 00), unsubscribe/privacy links, copyright.
- `heading()` — serif H1, `card()` — bordered info box with optional title, `detailRow()` — label/value row inside a card, `button()` — table-based brand pill CTA button (Outlook-safe), `stepsList()` — numbered lime-circle steps list, `agencyCard()` — logo + name + rating + jobs-completed card.

The 4 templates currently render against one shared dummy dataset (`lib/emails/dummy.ts`) so the whole customer journey previews consistently: a fictional lead `KOB-2026-4821` (Ingrid, moving from Thorvald Meyers gate 44 (Grünerløkka) to Ekebergveien 128 (Nordstrand), 12 September 2026, 3-room ~68m² apartment, 3rd floor no elevator → 1st floor short carry, wants flyttehjelp+flyttevask+pianotransport), and one fictional offer (14 200 kr fastpris from agency "Flyttefoten", chosen specifically because its logo is a PNG — SVG logos don't render reliably in most email clients, a detail worth preserving when picking which agency asset format to standardize on in Django).

Rebuild implication: because the reference numbering (`KOB-2026-{n}`), the detail rows (fra/til/dato/bolig/etasje/tjenester), and the offer fields (pris/omfang/inkludert/gyldig) are already fully modeled in these templates, they double as a near-complete spec for the `MoveLead` and `Offer` Django models needed to actually populate them.

13 Full data catalogs

Reproduce these as Django fixtures / seed data / an admin-editable model — copied verbatim from source so no content is lost in the port.

13.1 Cities (lib/cities.ts)

Slug	Name	Lat	Lon	Default map zoom
oslo	Oslo	59.9139	10.7522	11
bergen	Bergen	60.3913	5.3221	11
trondheim	Trondheim	63.4305	10.3951	11
stavanger	Stavanger	58.9700	5.7331	11
tromso	Tromsø	69.6492	18.9553	11

13.2 Oslo districts (lib/districts.ts) — 14 total

Each has: slug, name, lead (one-line hero subhead), body (unique descriptive paragraph for SEO uniqueness), metaDescription. Slugs: alna, bjerke, frogner, gamle-oslo, grorud, grunerlokka, nordre-aker, nordstrand, sagene, st-hanshaugen, stovner, søndre-nordstrand, ullern, vestre-aker, ostensjo — full copy for all 14 is in the source file lib/districts.ts (each paragraph is genuinely distinct, referencing real sub-neighborhoods, so treat this as real content to migrate, not filler).

13.3 Agencies (lib/agencies.ts) — 4 total

Slug	Name	Tagline	Rating	Reviews	Jobs done	Member since
loft	LØFT	Flytting med hodet, ikke bare ryggen	4.9	87	312	2023
relok	relok.	Hele Østlandet, hele veien	4.7	214	1840	2022
flyttefoten	Flyttefoten	Familiedrevet siden 2014	4.8	63	428	2024
flytteby	Flytteby	Nøkkelferdig flytting fra dør til dør	4.6	129	764	2021

Each agency record additionally carries: a logo asset path (one, relok, needs mix-blend-mode:multiply to render on light backgrounds — flag this per-logo in the Django model as a boolean, exactly as the source does with logoBlendMultiply), a 2–3 paragraph "about us" in first person, response-time string, a services list (5–6 items each), a coverage-areas list (6 items each), a contact object (name, role, phone — used specifically in the booking-confirmed email), and 4 written customer reviews each (name, ISO date, 1–5 stars, service type, full comment text). All of this is real, specific, non-generic copy in the source — migrate it verbatim, it's not lorem-ipsum placeholder.

13.4 Blog articles (lib/blog.ts) — 3 total

Slug	Title	Date	Read time
hva-er-viktig-a-tenke-pa-nar-du-skal-flytte	Hva er viktig å tenke på når du skal flytte?	2026-08-04	6 min
hva-koster-det-a-bruke-flyttebyrå-i-2026	Hva koster det å bruke flyttebyrå i 2026?	2026-07-21	5 min
flyttevask-dette-ma-vaere-gjort-for-du-leverer-nøklerne	Flyttevask: dette må være gjort før du leverer nøklene	2026-06-30	7 min

Each is a full, genuinely useful long-form guide (moving-day checklists, 2026 price ranges by home size, a pre-handover cleaning checklist) built from the typed block model in §10.2 — not placeholder text. One article's header image is a local asset (`boxes-and-plants.jpg` used inline); the header/hero images for all 3 articles are hotlinked Unsplash URLs (`images.unsplash.com/photo-...`) — these need to be downloaded and re-hosted (or replaced with licensed/local assets) for the Django rebuild, since hotlinking a third party's CDN in production is fragile and against most stock-photo licenses at scale.

14 SEO / sitemap behavior

`app/sitemap.ts` generates `/sitemap.xml` covering: the 5 static pages (home, wizard, partner, blogg, byraer), all 5 city pages, all 14 district pages, all 4 agency pages, and all 3 blog articles (with real `lastModified` dates on articles only). Priorities: home=1.0, other static=0.8, cities=0.7, districts=0.6, agencies=0.6, articles=0.5.

The sitemap's `BASE` constant is hardcoded to `https://kobly.vercel.app` — a preview/staging URL, not a production custom domain. Whatever the Django app's real domain is, this needs to be an environment-driven setting, not a literal string, from day one.

Every dynamic route (city, district, agency, article) generates its own `<title>` and meta description via `generateMetadata()`, all following consistent per-type templates (see each section above for the exact string patterns) — reproduce this as Django's `{% block title %} /meta tags per template, driven by the same model fields.`

15 Known quirks & open decisions

Non-obvious things found while reading the source that are worth a deliberate decision, not silent copying, when rebuilding:

- **No backend anywhere.** Wizard submit is `console.log`; email templates render but never send; there's no lead/offer/booking database. This is the headline finding — see §5.2 and §1.
- **Postal-code input doesn't resolve to an address.** `PostnummerInput` validates a 4-digit postcode client-side then does `/wizard?fra={code}` — the wizard drops that raw digit string straight into the free-text "Fra adresse" field with zero lookup (§4.4 note). Decide whether to preserve this low-fidelity behavior or actually resolve it server-side.
- **Orphaned components.** `components/wizard/Illustration.tsx` (6 hand-drawn SVG step icons) and `components/wizard/RadioCard.tsx` (an alternate selection-card pattern) exist in the codebase but are *not imported or rendered anywhere* — leftovers from an earlier design pass. Don't port them unless you specifically want that visual style; the current wizard uses `PillButton` / `BlockCard` defined inline in `page.tsx` instead.
- **Agency profile CTA doesn't pass agency context to the wizard.** "Få tilbud fra {agency} og 2 andre byråer" links to a plain `/wizard` with no query param identifying which agency was viewed — so the promise in that button's own label (that this specific agency is one of the three) isn't actually enforced anywhere in the flow. Decide whether the Django wizard should accept a preferred-agency hint.
- **Footer links to the internal email-preview tool.** `/email-preview` is meant as an internal QA page (`robots: noindex`) but is linked from the public production `Footer` component on every page. Decide whether to keep it public, gate it behind Django admin/staff auth, or drop the footer link and access it by direct URL only.
- **Partner "apply" flow is just a mailto link.** No partner-application database/form exists — `PartnerCTA` is literally `mailto:partner@kobly.no?subject=...`. If partner self-service onboarding is in scope for Django, this needs a real model + form, not just a port.
- **Sitemap base URL is a hardcoded preview domain** (`kobly.vercel.app`) — must become an environment setting.
- **Blog header images are hotlinked from Unsplash** — re-host or replace before going to production under a different domain/ownership.
- **District pages only exist for Oslo.** The other 4 cities (Bergen, Trondheim, Stavanger, Tromsø) have no district-level pages/routes — only Oslo does. If parity across cities is wanted later, that's new content work (14 unique district write-ups per city), not a code gap.

16 Django rebuild architecture & build order

16.1 Suggested build order (matches the stated priority)

1. **The wizard form end-to-end** (§5): templates/CSS/JS for all 5 steps + the map/autocomplete integration, a `MoveLead` model, server-side validation mirroring §5.5–5.9's rules, file upload storage for photos, and a working submit → persist → (at minimum) send the `receipt` email using the template from §12. This alone makes the site "functional" as a lead-capture tool.
2. **Agency data + matching stub**: an `Agency` model seeded from §13.3, and — even if real matching logic comes later — at least a manual/admin way to associate a `MoveLead` with agencies so the `new-lead` email can fire.
3. **Marketing shell**: base template with Header/Footer/MobileMenu (§4), then home page (§6), then the city/agency/blog pages, which are largely read-only content views over the §13 data catalogs.
4. **Offer/booking flow** (§12's `offer` and `booking-confirmed` emails imply an agency-facing "submit a quote" interface and a customer-facing "accept an offer" interaction) — this is genuinely new product surface with no existing UI to copy at all, only the resulting email as a spec of what data it must contain.
5. **Partner application flow** (§11) — currently just a mailto link; build a real form/model only if self-service partner signup is actually wanted, otherwise the mailto link ports as-is.

16.2 Suggested Django app boundaries

```
leads/      MoveLead, LeadImage, MoveLeadStatus — the wizard's backend (§5)
agencies/   Agency, Review, Offer, Booking — §13.3 + the offer/booking flow (§12)
content/    City, District, Article — §13.1/13.2/13.4, mostly read-only admin-editable content
partners/   PartnerApplication — new, if built (§11 / §15)
emails/     Port lib/emails/* almost verbatim as Django templates or Python string-builder
            functions; the 4 templates' HTML/table structure needs no redesign, only a
            templating-language swap and real data instead of lib/emails/dummy.ts.
```

16.3 Wizard-specific implementation notes

- **Multi-step UI without React** — two viable patterns: (A) one template, all 5 `<section>`s rendered at once, a small vanilla-JS controller toggles a `data-step` attribute and drives the same 260ms CSS keyframes, one hidden-input-mirrored JS state object posts everything at once on final submit — closest match to the current no-reload, animated feel; (B) a real Django `django-formtools SessionWizardView`, one URL per step, session-backed (gets free resume-after-refresh, which the current app doesn't have) at the cost of rebuilding the transition animation via CSS view-transitions or accepting plain page loads. Given the priority is fidelity to this exact UX, (A) is the closer port; (B) is more Django-idiomatic if a different feel is acceptable.
- **Server-side validation is not optional** — re-implement every rule in §5.5–5.9 (min lengths, phone regex, email regex, date-or-flexible) in `forms.py`. The existing client-side checks only gate a disabled button; they must never be trusted as the source of truth for a Django rebuild.
- **Map + autocomplete port near-verbatim** — Leaflet is framework-agnostic already; the current implementation is imperative Leaflet calls wrapped in React refs/effects, so removing the React lifecycle wrapper leaves genuine, reusable vanilla-JS map logic. The Geonorge fetch calls in §5.13 need only a hand-rolled debounce (`setTimeout / clearTimeout`) + `AbortController`, no framework.
- **File uploads** — switch from in-memory `File[]` + blob previews to a real `<input type="file" multiple>` inside `multipart/form-data`; use the `DataTransfer` API (or a simple "removed filenames" client-side filter) to let users still remove a specific file before submit, matching the current UX.

Generated from a direct, exhaustive read of the Kobly Next.js source — all 10 routes, ~34 components, 7 lib data modules, and 4 email templates — not from screenshots or guesswork.